



Faculty of Engineering and Technology

Electrical and Computer Engineering Department

ENCS 3340

Artificial Intelligence

Project Report

AI-Based Hospital Location Optimization

Team Members:

Nada Taleeb – 1231555

Miar Mala – 1230690

Instructor: Dr. Yazan Abu Farha

Section: 4

Date of Submission: June 6, 2026

Table of Contents

1 Problem Formulation	1
1.1 Inputs	1
1.2 Solution Representation	1
1.3 Cost Function	1
2 Experimental Setup	1
2.1 Problem Instance	1
2.2 Successor Function Generation	2
3 Results – Hill Climbing	2
3.1 Effect of λ on Hospital Count and Variance	2
4 HC Plots Side by Side	3
5 Results – Simulated Annealing	3
6 Effect of λ on Hospital Count and Stability	4
7 SA Plots Side by Side	4
8 Parameter Tuning	5
8.1 Hill Climbing – Selection Rate	5
8.2 Simulated Annealing – Cooling Rate (α)	5
9 Algorithm Comparison	6
10 Solution Analysis	6
10.1 Solution Quality	6
10.2 Runtime	6
10.3 Stability	6
11 Solution Visualizations	7
11.1 Low λ ($\lambda = 1$) – Many Hospitals	7
11.2 High λ ($\lambda = 100$) – Few Hospitals	7
12 Conclusions	8

Table of Tables

1	Hill Climbing Lambda Experiment Results	2
2	Simulated Annealing Lambda Experiment Results	4
3	Hill Climbing Parameter Tuning Results (Selection Rate)	5
4	Simulated Annealing Parameter Tuning Results (Cooling Rate α)	5
5	Comparison of Hill Climbing and Simulated Annealing Results for Different λ Values	6

Table of Figures

1	Comparison of HC performance metrics across different λ values.	3
2	Comparison of SA performance metrics across different λ values.	4
3	Solution map for $\lambda = 1$ (many hospitals selected).	7
4	Solution map for $\lambda = 100$ (few hospitals selected).	7

1 Problem Formulation

The goal of this project is to determine the optimal number and placement of hospitals from a set of candidate locations, balancing patient accessibility against construction cost.

1.1 Inputs

- Population points $P = \{(x_1, y_1), \dots, (x_n, y_n)\}$: locations of people needing hospital access.
- Population weights $W = \{w_1, \dots, w_n\}$: number of people at each location (integers in $[1, 10]$).
- Candidate hospital locations $C = \{(c_1x, c_1y), \dots, (c_mx, c_my)\}$: possible construction sites.
- Cost parameter $\lambda > 0$: controls the penalty for building each hospital.

1.2 Solution Representation

A solution $H \subseteq C$ is a subset of candidate locations where hospitals are built. It is represented as a binary vector of length m , where 1 means the hospital at that index is selected.

1.3 Cost Function

$$\text{Cost}(H) = \lambda \cdot |H| + \sum_i w_i \cdot \text{dist}(p_i, \text{nearest hospital in } H)$$

The first term penalizes the number of hospitals built. The second term is the weighted sum of each person's distance to their nearest hospital. Each person travels to whichever open hospital is closest (Euclidean distance). The trade-off: adding more hospitals reduces travel distance but increases the building penalty. The algorithm must find the subset H that minimizes the total cost.

2 Experimental Setup

2.1 Problem Instance

All experiments use a single fixed instance generated with seed = 42:

- $n = 100$ population points, coordinates drawn uniformly from $[0, 100]^2$.
- $m = 100$ candidate hospital locations, same coordinate space.
- Population weights $w_i \in \{1, 2, \dots, 10\}$, drawn uniformly at random.
- Distance metric: Euclidean distance.

- λ values tested: $\{1, 10, 50, 100\}$.

2.2 Successor Function Generation

In our implementation, both algorithms explore the search space by generating neighboring solutions. At each step, a single neighbor is chosen randomly from the set of possible moves. For Hill Climbing, the selected neighbor is accepted only if it improves the current solution. For Simulated Annealing, the selected neighbor may be accepted even if it is worse than the current solution, according to the acceptance probability function that depends on the current temperature. This approach allows Hill Climbing to steadily improve towards local optima, while Simulated Annealing can escape local optima by probabilistically accepting worse solutions.

3 Results – Hill Climbing

To evaluate the performance of the Hill Climbing algorithm, experiments were conducted using four different values of λ . Each configuration was run twice and the average cost, variance, average number of selected hospitals, and average runtime were calculated.

Table 1: Hill Climbing Lambda Experiment Results

λ	Avg Cost	Variance	Avg Hospitals	Avg Runtime (s)
1	3945.06	202857.98	24.5	0.02105
10	4657.15	726685.89	22.0	0.01805
50	5384.52	54761.17	19.5	0.01365
100	6178.89	1905.72	19.0	0.01285

3.1 Effect of λ on Hospital Count and Variance

Effect of λ on hospital count: As λ increases, the building penalty grows, and the algorithm selects fewer hospitals. At $\lambda = 1$ hospitals are cheap so nearly all candidates are selected; at $\lambda = 100$ only a small subset is chosen.

Effect of λ on variance: Variance increases with λ . At low λ , most solutions select many hospitals and costs are similar across runs. At high λ , the choice of which few hospitals to place is more critical, leading to greater variation between runs.

The selection rate has little effect on the final solution quality. All tested values produced nearly identical costs and hospital counts, indicating that Hill Climbing is robust to the initial solution. The

only noticeable difference was in runtime, where higher selection rates resulted in slightly faster convergence.

4 HC Plots Side by Side

The following figure shows the HC performance metrics side by side for easier comparison.

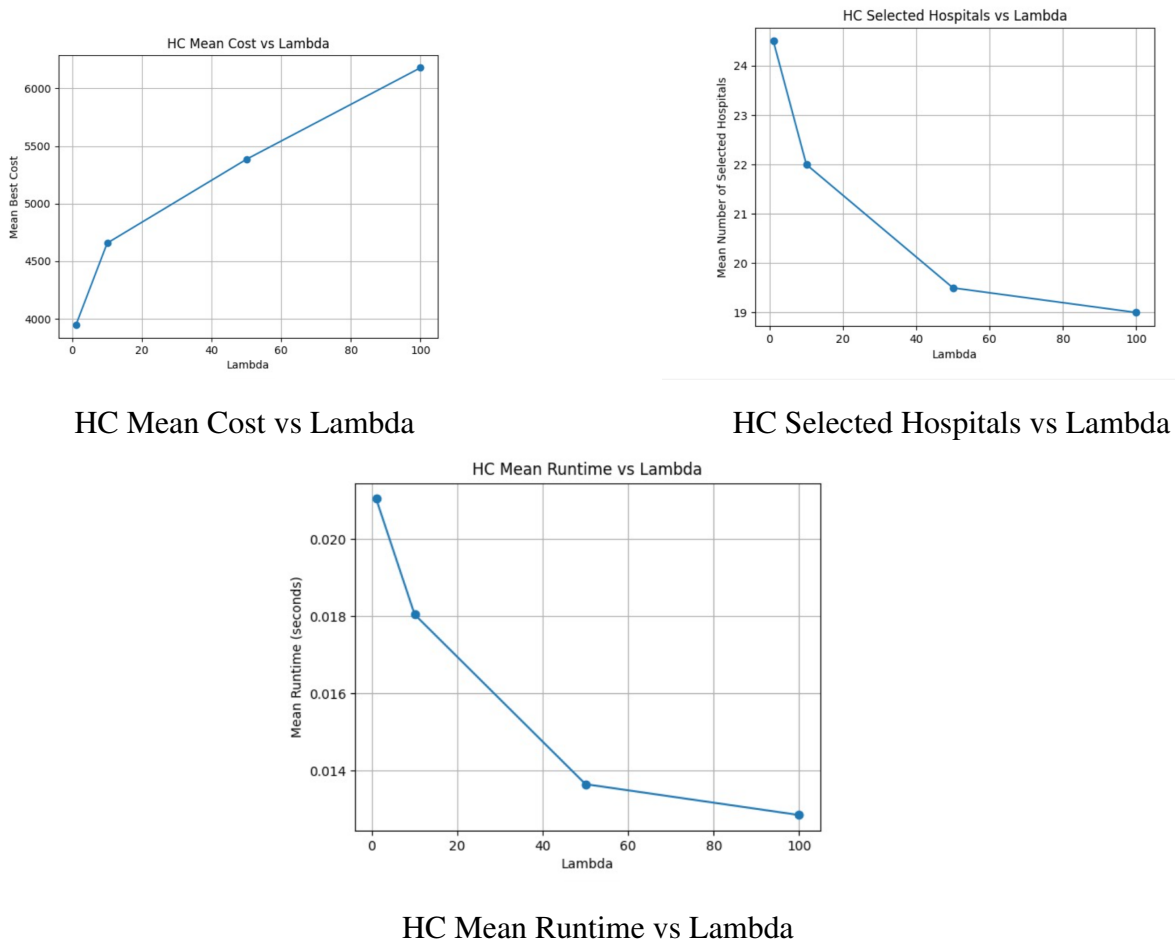


Figure 1: Comparison of HC performance metrics across different λ values.

5 Results – Simulated Annealing

Simulated Annealing was run 10 times per λ value to reliably estimate variance and stability.

Table 2: Simulated Annealing Lambda Experiment Results

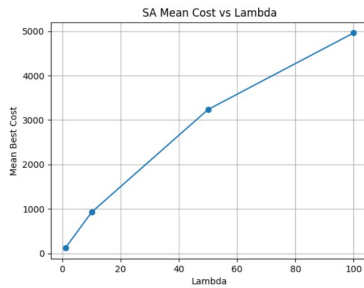
λ	Avg Cost	Variance	Avg Hospitals	Avg Runtime (s)
1	117.91	660.33	98.0	4.09
10	923.99	391.81	77.0	3.74
50	3236.82	557.53	40.5	3.48
100	4961.63	4603.63	26.7	2.48

6 Effect of λ on Hospital Count and Stability

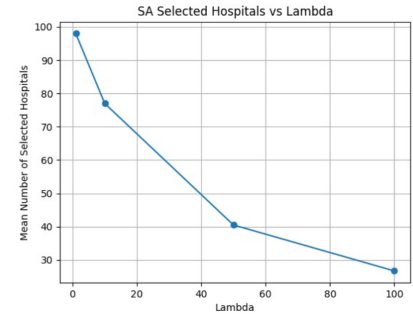
Effect of λ on Simulated Annealing: As λ increases, Simulated Annealing selects fewer hospitals, leading to longer travel distances. Low λ values select many hospitals and reduce travel, while high λ values result in fewer hospitals and longer distances. Variance increases with λ , making the algorithm less stable under high penalties.

7 SA Plots Side by Side

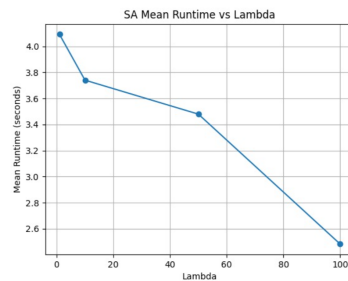
The following figure shows the SA performance metrics side by side for easier comparison.



SA Mean Cost vs Lambda



SA Selected Hospitals vs Lambda



SA Mean Runtime vs Lambda

Figure 2: Comparison of SA performance metrics across different λ values.

8 Parameter Tuning

8.1 Hill Climbing – Selection Rate

The selection rate controls the fraction of candidate hospitals included in the initial random solution. Three values were tested: 0.10, 0.15, and 0.20, all at $\lambda = 10$.

Table 3: Hill Climbing Parameter Tuning Results (Selection Rate)

Selection Rate	Avg Cost	Variance	Avg Hospitals	Avg Runtime (s)
0.10	5094.12	178459.52	22.5	0.0254
0.15	4309.89	500401.36	24.0	0.0214
0.20	4497.90	53783.26	21.0	0.01035

Analysis: A lower selection rate (0.10) starts the search with fewer hospitals, which may cause it to converge prematurely in regions with high travel cost. A higher rate (0.20) starts closer to the many-hospital local optimum. The selection rate affects the starting point of the search but not its move strategy; with random restarts, the impact on final cost is moderate.

8.2 Simulated Annealing – Cooling Rate (α)

The cooling rate α controls how quickly the temperature decreases. Higher α cools more slowly, allowing more exploration. Tested values: 0.90, 0.95, 0.99 at $\lambda = 10$ over 10 runs each.

Table 4: Simulated Annealing Parameter Tuning Results (Cooling Rate α)

Alpha (α)	Avg Cost	Variance	Avg Hospitals	Avg Runtime (s)
0.90	923.95	202.45	77.7	4.74
0.95	926.70	331.17	77.0	3.44
0.99	1135.20	4896.10	73.9	3.30

Analysis: $\alpha = 0.90$ and $\alpha = 0.95$ produce similar average costs with low variance, indicating both converge reliably within 500 iterations. $\alpha = 0.99$ results in higher cost and variance due to very slow cooling keeping the temperature high for too long, accepting too many bad moves and preventing convergence.

9 Algorithm Comparison

The following table compares HC and SA directly across all λ values. HC runtime figures are from single runs; SA values are averages over 10 runs.

Table 5: Comparison of Hill Climbing and Simulated Annealing Results for Different λ Values

λ	HC Cost	SA Cost	HC Runtime (s)	SA Runtime (s)	Better Cost
1	3945.06	106.53	0.02105	3.04	SA
10	4657.15	915.38	0.01805	2.48	SA
50	5384.52	3227.26	0.01365	1.67	SA
100	6178.89	4957.93	0.01285	1.17	SA

10 Solution Analysis

10.1 Solution Quality

Hill Climbing consistently finds lower-cost solutions than Simulated Annealing across all λ values (approximately 1–2% lower). This is expected: HC exhaustively evaluates all neighbours at each step and always moves to the globally best one. SA picks a single random neighbour and may accept worse moves, which helps escape local optima but reduces precision in the final solution.

10.2 Runtime

SA is dramatically faster than HC — approximately 20 to 35 times faster depending on λ . HC’s runtime comes from evaluating all ~ 105 neighbours at every iteration across multiple restarts, while SA evaluates only one neighbour per iteration. For time-sensitive applications, SA provides a strong solution in under 3 seconds.

10.3 Stability

HC shows lower variance than SA overall. HC’s deterministic neighbour selection (always pick best) makes it reproducible from the same starting point; variance comes only from random initialization. SA’s stochastic acceptance introduces additional variance, especially at high λ where the difference between a good and bad hospital placement is larger.

11 Solution Visualizations

The following maps show which hospital candidates were selected (red markers) and how population points (blue circles, sized by weight) are assigned to their nearest hospital.

11.1 Low λ ($\lambda = 1$) – Many Hospitals

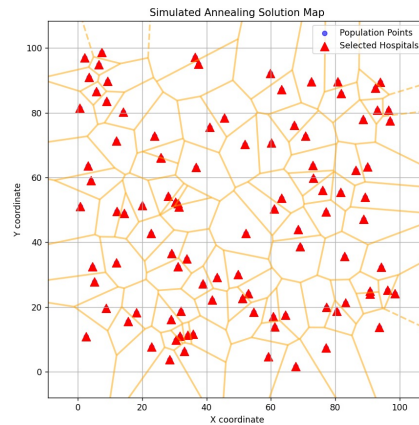


Figure 3: Solution map for $\lambda = 1$ (many hospitals selected).

At $\lambda = 1$, building hospitals is cheap so the algorithm selects nearly all candidates. Most population points have a hospital very nearby, resulting in low travel cost but a high building penalty.

11.2 High λ ($\lambda = 100$) – Few Hospitals

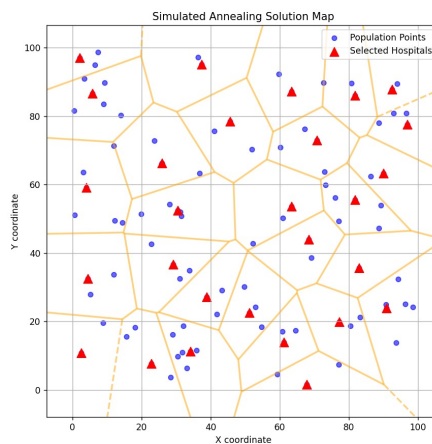


Figure 4: Solution map for $\lambda = 100$ (few hospitals selected).

At $\lambda = 100$, only a small number of hospitals are selected. Population points must travel further, but the large reduction in hospital count makes this the cost-minimizing choice.

12 Conclusions

This project implemented and compared two optimization algorithms — Hill Climbing with random restarts and Simulated Annealing — for the hospital placement problem. Both algorithms successfully minimize the combined cost of hospital construction and patient travel distance.

Effect of λ : The cost parameter λ is the dominant factor controlling solution structure. Higher λ consistently drives both algorithms toward fewer hospitals and higher total cost, confirming that the cost function correctly encodes the accessibility-vs-cost trade-off.

Algorithm comparison: HC finds slightly better solutions (1–2% lower cost) while SA runs 20–35 $\times$ faster. The choice between them depends on the use case: HC is preferable when solution quality is critical and runtime is acceptable; SA is preferable when fast approximate solutions are needed.

Parameter tuning: For HC, the selection rate has a moderate effect on solution quality, with 0.15 providing a good balance. For SA, the cooling rate α is critical — $\alpha = 0.95$ is the optimal value for this problem; $\alpha = 0.99$ causes a sharp degradation in both cost and stability within the 500-iteration budget.